

1. **Nick's Heavy Threads** Nick operates a store in Marketview Mall which has poor lighting, blasts black metal and sells jeans. Only one pair of jeans is available to purchase at a time, though there are more stored in the back. If a size is out that you don't want, you must wait for someone else to purchase the jeans. Nick's only employee, Hank, sits in a chair and stares at people angrily until someone makes a purchase, at which point he replaces the jeans with the same model of a random size. In order to prevent customers' waiting infinitely for an unavailable size, Hank will switch the jeans for a different size pair if no one has bought them after a period of three seconds.

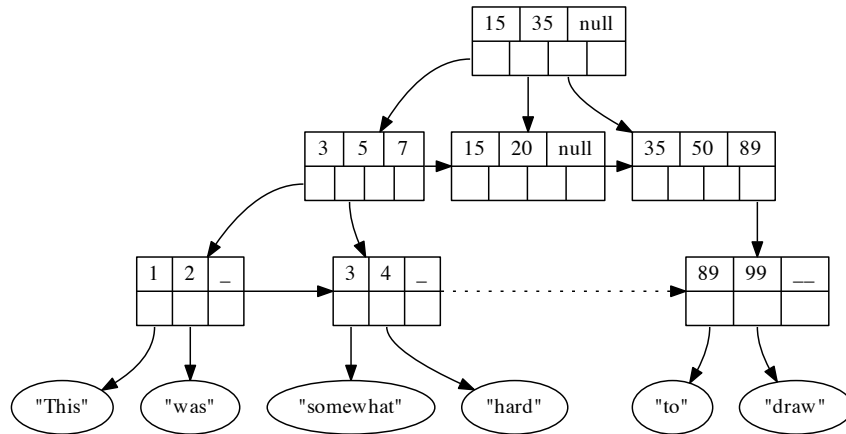
```
1 public class NicksHeavyThreads
2 {
3     private static int awesomeJeans = 0; // jeans' size [1-5], or 0 when none on display
4
5     private static int customers = 0; // keep this updated as customers arrive and leave
6
7     private static MeanWorker hank = new MeanWorker();
8
9     public static void main( String[] args )
10    {
11        for( int i = 0; i < 10; i++ )
12        {
13            ( new LameCustomer() ).start();
14        }
15        try { Thread.sleep( 1000 ); } // wait one second before introducing Hank
16        catch( InterruptedException pleaseDont ) {}
17        hank.start();
18    }
19
20    private static class LameCustomer extends Thread
21    {
22        // implementation omitted
23    }
24
25    private static class MeanWorker extends Thread
26    {
27        // implementation omitted
28    }
29 }
```

- (a) Complete the implementation of the LameCustomer class; each instance must choose a jeans size and wait for it to be available, update the jeans to indicate that they have been taken, print the message "Customer: I got my size *size* jeans!" and inform all threads that the jeans selection has changed. (Hint: Remember to keep an accurate count of how many customers are in the shop at any point.)

- (b) Now implement the `MeanWorker` class, which should choose a size and stock a pair of jeans of that size, print the message “Hank: I grumpily restocked with size *size*,” and inform all threads that the selection has changed. It should then wait until someone has taken the jeans or until three seconds have elapsed, whichever comes first. These steps should be repeated until all customers have left the store.

2. **B⁺Trees** A B⁺Tree is a data structure that can be used to map from keys to values. Figure 1 shows a tree that maps from `ints` to `Strings`.

Figure 1: A partially drawn B⁺Tree of strings, keyed on `ints`



- (a) What Java Collections Framework interface should a B⁺Tree be able to implement?
- (b) We need to implement two different kinds of nodes for the tree. Why do we need to do this? What is different between the two types of nodes?
- (c) We need classes to represent the nodes of the tree. Implement these classes so that they use generic types and all of their members are package private. Don't implement the constructors or any other methods.

3. What is the output when LookAtDatMagic's main is executed?

```
1 public class HeySteve{
2     public int bananza(int in) throws NewException{
3         if ( in == 7 ){
4             throw new NewException("HeySteve, cut that out!");
5         }
6         return in;
7     }
8 }
9
10 public class NewException extends Exception{
11     public NewException(String message){
12         super(message);
13     }
14 }
15 }
16
17 public class WakaWaka{
18     public String BeachBash(Object a, Object b) throws NewException{
19         if ( a.equals(b) ){
20             throw new NewException("It 's a Beach-bash! WakaWaka!");
21         }
22         return "Da-nanananan";
23     }
24 }
25
26 public class LookAtDatMagic{
27     public void magic() throws NewException{
28         int maraca = 5;
29         try{
30             HeySteve steve = new HeySteve();
31             maraca = steve.bananza(7);
32         }catch(NewException e){
33             System.out.println(e.getMessage());
34         }finally{
35             WakaWaka waka = new WakaWaka();
36             System.out.println(waka.BeachBash(maraca, 5));
37         }
38     }
39
40     public static void main(String[] args){
41         try{
42             LookAtDatMagic ladm = new LookAtDatMagic();
43             ladm.magic();
44         }catch(NewException e){
45             System.out.println(e.getMessage());
46         }
47     }
48 }
```

4. Briefly describe the difference between classes and objects.

5. Briefly describe the difference (for objects) between `a.equals(b)`, `a==b`, `a.compareTo(b)`, and `Comparator.compare(a,b)`.

6. Name the design pattern used in the following snippet of code.

```
1 public class Car
2 {
3     private String make;
4     private String model;
5     private int mileage;
6
7     public Car(String make, String model, int mileage)
8     {
9         this.make=make;
10        this.model=model;
11        this.mileage=mileage;
12    }
13    public static Car makeCar(String str)
14    {
15        String arr[] = str.split(" ");
16        return new Car(arr[0], arr[1], Integer.parseInt(arr[2]));
17    }
18
19    public static void main(String[] args)
20    {
21        Car myCar = Car.makeCar("Toyota Camry 200000");
22    }
23 }
```

7. Which Layout Manager do each of the following descriptions describe?

- (a) Displays the contained components in a set number of rows and columns, where each row has the same height and each column the same width.
- (b) Attempts to display the contained components in a single row but will create a new row if necessary.
- (c) Allows the developer to specify the region (North, South, East, West, Center) of the panel to place the component in.

8. (a) What is the Java Collections Framework?

(b) Assume the following line of code is given:

```
1      Collection<Integer> t = new ArrayList<Integer>();
```

What then is wrong with the following? Correct any errors:

```
1      for( int i = 0; i < 20; i++){
2          t.add(i);
3      }
4      for( int i=0; i < t.size(); i++){
5          System.out.println(t.get(i));
6      }
```

9. Given the following class, Write the code necessary for the main to run.

```
1 public class UFO{
2     private Collection<Probable> toProbe;
3     private int aggression;
4
5     public UFO(int agg){
```

```

6         toProbe = new ArrayList<Probable>();
7         aggression = agg;
8     }
9
10    public void probeEverything(){
11        for( Probable p : toProbe ){
12            p.probe();
13        }
14    }
15
16    public void sendHome(){
17        for( Probable p: toProbe ){
18            p.returnHome();
19        }
20    }
21
22    public void abduct(Collection<Probable> potentialAbductees){
23        for( Probable p : potentialAbductees ){
24            if( p.getMentalResolve() < aggression){
25                toProbe.add(p);
26            }
27        }
28    }
29
30    public static void main(String[] args){
31        UFO ufo = new UFO(1199999999);
32
33        field.add(new Cow("Bessie"));
34        //10 = mentalResolve
35        Human cleatus = new RedNeck("Cleatus", 10));
36
37        // 50 == mentalResolve and 100 == academicRespect
38        Human brown = new Professor("Brown", 50, 100));
39
40        field.add(cleatus);
41        field.add(brown);
42
43        ufo.abduct(field);
44        ufo.probeEverything();
45        ufo.sendHome();
46    }
47 }

```


How was our review? Please let us know by filling out our survey at
<http://clipboard.rit.edu>, survey id: FCF9EA34